

ТЕХНИЧЕСКОЕ ЗАДАНИЕ

Проект «Цифровой двойник локомотива» для хакатона КТЖ

Версия 2.0

04.04.2026

Параметр	Значение
Заказчик	Хакатон КТЖ
Назначение	Рабочее ТЗ для команды разработки
Локомотивы	KZ8A (электровоз), ТЭ33А (тепловоз)
Стек	React + Next.js / FastAPI / TimescaleDB / Redis
Источник логики	Locomotive_Metrics_Reference.md

Документ фиксирует архитектурные, функциональные и нефункциональные требования.
Во всех местах, не определённых заказчиком жёстко, выбран наиболее подходящий инженерный вариант.

Содержание

1	Общая цель проекта	4
2	Контекст и исходные предпосылки	4
2.1	Критерии соответствия безопасности	4
3	Критерии оценки хакатона	4
4	Поддерживаемые типы локомотивов	5
5	Стек и принятые технические решения	5
6	Пользовательские роли и аутентификация	5
6.1	Роли	5
6.2	Аутентификация и авторизация	6
7	Функциональные требования	6
7.1	Обязательные экраны MVP	6
7.2	Login Page	6
7.3	Main Dashboard	6
7.4	Alerts Panel	6
7.5	Карта маршрута / Схема пути	7
7.6	Trends / Charts	7
7.7	Replay / History	7
7.8	Report Export	7
7.9	Admin Settings (только роль Admin)	7
7.10	Адаптивность и доступность	7
8	Архитектура системы	8
8.1	Компоненты	8
8.2	Архитектурная диаграмма	8
8.3	Логи, метрики, health-check	9
9	Обработка и хранение данных	9
9.1	Data Pipeline	9
9.2	Схема TimescaleDB	9
9.3	Retention и агрегация	10
10	Индекс здоровья — математическая модель	11
10.1	Обзор	11
10.2	Функция нормализации	11
10.3	Штрафы за алерты	11
10.4	Оценка подсистемы	11
10.5	Глобальный Health Index	12
10.6	Категоризация	12
10.7	Объяснимость — Top-5 факторов	12
10.8	Частота и производительность	12

11 Спецификация REST API	12
11.1 Общие соглашения	12
11.2 Authentication	12
11.3 Telemetry	13
11.4 Health Index	13
11.5 Alerts	13
11.6 Configuration	13
11.7 Simulator Control	14
11.8 Export	14
11.9 System	14
11.10 WebSocket Protocol	14
12 Описание экранных форм	15
12.1 Login Screen	15
12.2 Main Dashboard	15
12.3 Alerts Panel	15
12.4 Route Map	15
12.5 Trends / Charts	15
12.6 Replay / History	15
12.7 Admin Settings	15
13 Симулятор телеметрии	16
13.1 Normal Operation Mode	16
13.2 Аварийные сценарии (минимум 3)	16
13.3 Connection Loss	16
13.4 Конфигурация	16
14 Нефункциональные требования	16
14.1 Производительность / Latency	16
14.2 Highload / Burst	16
14.3 Отказоустойчивость	17
14.4 Безопасность	17
14.5 Документация API	17
14.6 Инфраструктура	17
14.7 Логи и наблюдаемость	17
15 План тестирования	17
15.1 Unit Tests	17
15.2 Integration Tests	18
15.3 Scenario / E2E Tests	18
15.4 Performance Tests	19
16 Формат сдачи и демонстрация	19
16.1 Git-репозиторий	19
16.2 Презентация	20
16.3 Демонстрация	20
16.4 Архитектурная диаграмма	20
17 Definition of Done	20
18 Глоссарий	21

19 Приложения

21

1 Общая цель проекта

Разработать **production-like full-stack прототип** веб-приложения «Цифровой двойник локомотива», которое в режиме, приближённом к реальному времени:

- принимает поток телеметрии от бортовых систем локомотива;
- рассчитывает интегральный **Health Index** (0–100) с объяснимостью (top-5 факторов);
- выявляет опасные отклонения и формирует алерты с рекомендациями;
- предоставляет машинисту, диспетчеру и администратору понятный интерфейс мониторинга, диагностики и анализа.

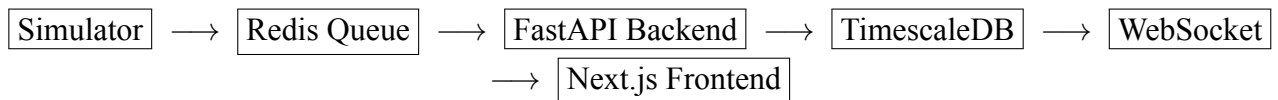
Основной приоритет — безопасность локомотива, безопасность движения и снижение риска для окружающей среды.

При распределении усилий команда должна ориентироваться на **весовые критерии оценки** (см. раздел 3): наибольший вес имеет работа в реальном времени (35 %) и дизайн интерфейса (30 %).

2 Контекст и исходные предпосылки

Реальные производственные данные КТЖ команде **недоступны**. Решение реализуется как реалистичный демонстрационный прототип, в котором бортовой источник заменяется *lightweight simulator service*.

Демонстрационный пайплайн:



Все технические характеристики локомотивов и логика оценки параметров берутся из файла `Locomotive_Metrics_Reference.md`.

2.1 Критерии соответствия безопасности

- Не использовать реальные коммерческие данные без согласования.
- Обезличивание в примерах; mock-данные по умолчанию.
- Конфигурационные секреты — **только через переменные окружения** (ENV).

3 Критерии оценки хакатона

#	Критерий	Вес	Что оценивается
1	Дизайн интерфейса (UI/UX)	30 %	Читабельность, приоритизация сигналов, информативные визуализации, accessibility, темизация.
2	Работа в реальном времени	35 %	WebSocket/SSE, стабильность под нагрузкой, задержки, отказоустойчивость клиента и сервера.
3	Архитектура backend	25 %	Чистая модульность, масштабируемость, хранение истории, прозрачность индекса здоровья.
4	Демонстрация и инженерная культура	10 %	Чистота репозитория, инструкции запуска, логи/метрики, тестовый сценарий.

4 Поддерживаемые типы локомотивов

Система обязана поддерживать **два типа** локомотивов:

Тип	Тяга	Особенности
KZ8A	Электровоз	Переменный ток 25 кВ, тяговые двигатели, рекуперативное торможение
ТЭЗ3А	Тепловоз	Дизель-электрическая, дизельный двигатель + генератор, расход топлива

Для каждого типа существует **отдельный конфигурационный профиль**: список параметров, единицы измерения, безопасные/предупредительные/критические диапазоны, группировка по подсистемам, веса подсистем. Переключение типа локомотива **не требует изменения кода** — используется конфигурационный подход.

Конкретные значения параметров и диапазонов — см. *Locomotive_Metrics_Reference.md*.

5 Стек и принятые технические решения

Слой	Технология	Комментарий
Frontend	React + Next.js	App Router, TypeScript, SSR для начального рендера
UI	Tailwind CSS + shadcn/ui	Быстрый стек с поддержкой темизации
Charts	ECharts	Высокопроизводительные интерактивные графики
Maps	Leaflet / MapLibre	Карта маршрута с маркерами и оверлеями
Backend	FastAPI (async)	Python 3.11+, асинхронные endpoints и WebSocket
Очередь	Redis / asyncio.Queue	Шина событий, буферизация потока телеметрии
Database	TimescaleDB	PostgreSQL + расширение для time-series, hypertables
Realtime	WebSocket (SSE fallback)	Поток обновлений backend → frontend
Infra	Docker Compose	Единый способ запуска всех сервисов
API Docs	OpenAPI / Swagger	Авто-генерация на /docs
Logging	structlog (JSON)	Структурированные логи, trace_id

6 Пользовательские роли и аутентификация

6.1 Роли

Роль	Описание	Доступ к настройкам
Admin	Управление конфигурацией, порогами, сценарием симуляции, пользователями. Полный доступ ко всем экранам + панель администрирования.	Полный
Dispatcher	Основной режим мониторинга: dashboard, alerts, trends, replay/history, export. Не может изменять пороги.	Только чтение
Cabin mode	Упрощённый UI-режим для машиниста. Крупный Health Index, только ключевые показатели, высокий контраст.	Нет доступа

6.2 Аутентификация и авторизация

- **Механизм:** JWT-аутентификация (JSON Web Token).
- **Login screen:** форма с полями username/password.
- **Демо-пользователи:** предзаданные учётные записи в переменных окружения (`.env`):
 - admin / admin123 — роль Admin
 - dispatcher / disp123 — роль Dispatcher
 - cabin / cabin123 — роль Cabin
- **Защита маршрутов:** frontend проверяет роль перед рендером; backend — middleware на каждом endpoint.
- **Ограничение доступа:** Admin-only endpoints возвращают **403 Forbidden** для других ролей.

7 Функциональные требования

7.1 Обязательные экраны MVP

Login Page, Main Dashboard, Alerts Panel, Trends / Charts, Subsystem Cards, Карта маршрута, Replay / History, Report Export, Admin Settings.

7.2 Login Page

Центрированная карточка: логотип, поля username и password, кнопка «Войти». При ошибке — toast-уведомление. Автоматический редирект при наличии валидного токена.

7.3 Main Dashboard

- **Health Index виджет** — крупный круговой индикатор (0–100), цветовая индикация статуса (Normal / Attention / Critical), текстовая подпись.
- **Топ-5 факторов ухудшения** — список параметров с наибольшим вкладом в снижение индекса. Для каждого: название, подсистема, текущее значение vs порог, полоса вклада.
- **Панель алертов** — последние 10 алертов, иконка severity + время + сообщение, прокрутка.
- **Рекомендуемые действия** — текстовый список, формируемый на основе активных алертов.
- **Панели ключевых узлов:**
 - **Скорость** — текущая, предел, мини-график.
 - **Топливо / Энергия** — уровень/расход, тренд.
 - **Давления / Температуры** — ключевые показатели с цветовой индикацией.
 - **Электрика** — напряжение, ток, статус цепей.
- **Селектор локомотива** — выпадающий список (KZ8A / ТЭ33А).
- Автообновление через WebSocket; визуальный индикатор «нет связи» при потере соединения.

7.4 Alerts Panel

- Таблица алертов: столбцы — Время, Параметр, Severity (иконка + текст), Значение, Порог, Статус (new/acknowledged), Аннотация.
- Фильтры: чекбоксы severity, диапазон времени, селектор локомотива.

- Клик по строке — разворачивает детали + поле для **отметки события** (аннотации): текстовое поле для ввода комментария к алерту/событию.
- Кнопка «Acknowledge» для подтверждения прочтения алерта.
- Пагинация.

7.5 Карта маршрута / Схема пути

- Упрощённая карта или схема участка пути (Leaflet/MapLibre).
- Маркер текущего положения локомотива.
- Наложение действующих скоростных ограничений (зоны на карте).
- Цветовая кодировка сегментов маршрута по состоянию Health Index.
- Overlay-панель: текущая скорость, координаты, ближайшая станция.
- Маршрут задаётся через GeoJSON файл в конфигурации.

7.6 Trends / Charts

- Интерактивные графики по ключевым метрикам (ECharts).
- Настраиваемое окно просмотра: 5, 10, 15 минут + произвольный диапазон.
- Tooltip с точными значениями, авто-скейлинг оси Y.
- Zoom (drag-select), pan, overlay нескольких метрик.
- Корректная работа с потоком данных от 1 Гц и выше.
- Автообновление в реальном времени с возможностью паузы.

7.7 Replay / History

- Выбор временного диапазона (последние 5–15 минут).
- Timeline-скраббер (полоса перемотки) с кнопками Play / Pause.
- Управление скоростью воспроизведения: 1×, 2×, 5×.
- **Синхронное обновление** всех компонентов: графики, Health Index, алерты, карта пути — привязаны к позиции скраббера.

7.8 Report Export

- Экспорт мини-отчёта в формате **PDF** или **CSV**.
- Содержание отчёта: временной диапазон, сводка Health Index (min/avg/max), список сработавших алертов, средние значения ключевых метрик.
- Вызывается из экрана Replay или Dashboard.

7.9 Admin Settings (только роль Admin)

- Интерфейс с вкладками:
 - **Пороги** — таблица по типу локомотива: параметр, safe_low, safe_high, warn_low, warn_high, crit_low, crit_high, weight. Инлайн-редактирование.
 - **Симуляция** — кнопки Start/Stop, выпадающий список сценариев, индикатор статуса.
 - **Пользователи** — простая CRUD-таблица (username, role, сброс пароля).

7.10 Адаптивность и доступность

- Адаптивная вёрстка: корректное отображение на мониторах и ноутбуках различных разрешений.

- Обязательна **светлая и тёмная тема** с переключателем в header.
- Accessibility (WCAG AA): достаточный контраст, читабельный размер шрифта, текстовые tooltips на всех иконках, навигация клавиатурой для критичных элементов.

8 Архитектура системы

8.1 Компоненты

Simulator Service

Отдельный Python-процесс (или контейнер), генерирующий телеметрию для KZ8A и ТЭЗ3А. Поддерживает нормальные и аварийные сценарии. Публикует JSON-сообщения в Redis Queue с частотой 1 Гц на локомотив.

Шина событий (Redis / asyncio.Queue)

Легковесная очередь, отвязывающая simulator от backend. Буферизирует входящий поток для защиты от всплесков нагрузки. Гарантирует порядок FIFO.

FastAPI Backend

- Ingest worker: забирает сообщения из очереди, валидирует, нормализует, сглаживает.
- Health Index Engine: пересчитывает индекс на каждый тик.
- Alert Generator: проверяет пороги, создаёт алерты.
- REST API: исторические данные, конфигурация, экспорт.
- WebSocket Server: раздаёт live-обновления подключённым клиентам.

TimescaleDB

PostgreSQL с расширением TimescaleDB. Хранение time-series телеметрии (hypertable), истории Health Index, алертов, конфигураций, пользователей. Retention policy 24–72 часа.

Next.js Frontend

SPA на App Router + TypeScript. WebSocket-клиент с логикой переподключения. State management: React Context или Zustand. Рендеринг графиков через ECharts, карты через Leaflet.

8.2 Архитектурная диаграмма

В репозитории должна присутствовать архитектурная диаграмма (docs/architecture.png или .svg), отображающая:

- Все компоненты системы в рамках Docker Compose.
- Потоки данных: Simulator → Redis → FastAPI → TimescaleDB.
- Push-поток: FastAPI (WebSocket) → Browser.
- Pull-поток: Browser → FastAPI (REST) → TimescaleDB.
- Открытые порты (frontend: 3000, backend: 8000, DB: 5432).

8.3 Логи, метрики, health-check

- **Логи:** все backend-сервисы используют `structlog` (Python) для структурированных JSON-логов. Поля: `timestamp`, `level`, `component`, `message`, `trace_id`.
- **Уровни логирования:** `DEBUG` (dev), `INFO` (prod по умолчанию), `WARNING`, `ERROR`.
- **Health-check endpoint:** `GET /health` — возвращает статус сервиса, связность с БД, глубину очереди, `uptime`.
- **Метрики:** активные WebSocket-соединения, сообщений/сек (обработано), глубина очереди, время вычисления HI.

9 Обработка и хранение данных

9.1 Data Pipeline

Пошаговый поток обработки данных:

1. Simulator генерирует JSON-сообщение (`loco_id`, `loco_type`, `ts`, `parameters`).
2. Сообщение помещается в Redis Queue (или `asyncio.Queue`).
3. Ingest worker извлекает сообщение из очереди.
4. **Валидация:** проверка обязательных полей, типов значений, формата `timestamp`.
5. **Дедупликация:** отбрасываются сообщения с дублирующим (`loco_id`, `ts`).
6. **Фильтрация выбросов:** значения за пределами физически возможного диапазона отвергаются (диапазоны настраиваются `per parameter`).
7. **Обработка missing values:** `forward-fill` последним известным значением (максимум 5 с, далее `NaN`).
8. **Сглаживание:** `EMA` ($\alpha = 0.3$) или медианный фильтр (окно = 5). Выбирается `per parameter` в конфигурации.
9. Запись `raw` + `smoothed` значений в `telemetry_raw` (TimescaleDB).
10. Триггер пересчёта Health Index → запись в `health_index_history` → `push` через WebSocket.

9.2 Схема TimescaleDB

Таблица `telemetry_raw` (hypertable)

Поле	Тип	Описание
<code>ts</code>	<code>TIMESTAMPTZ NOT NULL</code>	Временная метка (partition key)
<code>loco_id</code>	<code>VARCHAR(32) NOT NULL</code>	Идентификатор локомотива
<code>loco_type</code>	<code>VARCHAR(16) NOT NULL</code>	Тип: KZ8A TE33A
<code>parameter</code>	<code>VARCHAR(64) NOT NULL</code>	Название параметра
<code>value_raw</code>	<code>DOUBLE PRECISION</code>	Сырое значение
<code>value_smooth</code>	<code>DOUBLE PRECISION</code>	Сглаженное значение (EMA / медиана)
<code>unit</code>	<code>VARCHAR(16)</code>	Единица измерения
<code>quality</code>	<code>SMALLINT DEFAULT 0</code>	0 = ok, 1 = interpolated, 2 = outlier

Chunk interval: 1 час. Retention policy: 72 часа (`drop_chunks`).

Индекс: (`loco_id`, `parameter`, `ts DESC`).

Таблица **health_index_history** (hypertable)

Поле	Тип	Описание
ts	TIMESTAMPTZ NOT NULL	Временная метка
loco_id	VARCHAR(32) NOT NULL	Идентификатор локомотива
score	DOUBLE PRECISION NOT NULL	Значение HI (0–100)
status	VARCHAR(16) NOT NULL	Normal Attention Critical
top_factors	JSONB	Массив top-5 факторов
subsystem_scores	JSONB	Оценки подсистем

Chunk interval: 1 час. Retention: 72 часа.

Таблица **alerts**

Поле	Тип	Описание
id	BIGSERIAL PK	Автоинкремент ID
ts	TIMESTAMPTZ NOT NULL	Время срабатывания
loco_id	VARCHAR(32) NOT NULL	Идентификатор локомотива
parameter	VARCHAR(64) NOT NULL	Параметр, вызвавший алерт
severity	VARCHAR(16) NOT NULL	info warning critical
value	DOUBLE PRECISION	Фактическое значение
threshold	DOUBLE PRECISION	Превышенный порог
message	TEXT	Текст алерта
acknowledged	BOOLEAN DEFAULT FALSE	Подтверждён ли
annotation	TEXT	Отметка события (аннотация)

Индекс: (loco_id, ts DESC, severity).

Таблица **loco_config**

loco_type	VARCHAR(16) PK	Тип локомотива
config	JSONB NOT NULL	Параметры, диапазоны, веса, подсистемы
updated_at	TIMESTAMPTZ	Время последнего обновления

Таблица **users**

id	SERIAL PK	ID пользователя
username	VARCHAR(64) UNIQUE	Логин
password_hash	VARCHAR(256)	Хеш пароля (bcrypt)
role	VARCHAR(16)	admin dispatcher cabin

9.3 Retention и агрегация

- Retention сырых данных: **72 часа** (настраивается через ENV).
- Для replay — прямые запросы к hypertable по временному диапазону.
- Опционально: continuous aggregate — 1-минутные средние для графиков с большим окном.

10 Индекс здоровья — математическая модель

10.1 Обзор

Health Index (HI) — единый числовой показатель $\in [0, 100]$, отражающий общее состояние локомотива. Расчёт **иерархический**:

$$\text{Параметры} \xrightarrow{\text{нормализация}} \text{Подсистемы} \xrightarrow{\text{взвешивание}} \text{Глобальный HI}$$

Индекс учитывает: текущие (сглаженные) значения, тренды, активные диагностические сигналы и штрафы за алерты.

10.2 Функция нормализации

Для каждого параметра p с текущим сглаженным значением v определяется нормализованная оценка $\text{norm}(v, p) \in [0, 1]$:

$$\text{norm}(v, p) = \begin{cases} 1.0, & \text{если } v \in [\text{safe_low}_p, \text{safe_high}_p] \\ 1 - \frac{|v - s_{\text{near}}|}{t_{\text{bound}} - s_{\text{near}}}, & \text{если } v \in \text{warning zone} \\ 0.0, & \text{если } v \notin [\text{crit_low}_p, \text{crit_high}_p] \end{cases}$$

где s_{near} — ближайшая граница safe-зоны, t_{bound} — соответствующая граница critical-зоны. Особые случаи: если $v = \text{NaN}$ (пропуск данных > 5 с), то $\text{norm} = 0$ (Critical).

Конкретные значения `safe_low`, `safe_high`, `warn`, `crit` для каждого параметра — см. `Locomotive_Metrics_Reference.md`.

10.3 Штрафы за алерты

Активные алерты накладывают мультипликативный штраф на оценку затронутой подсистемы:

$$\text{penalty}(\text{sub}) = \max(0.5, 1 - 0.1 \cdot N_{\text{crit}} - 0.05 \cdot N_{\text{warn}})$$

где N_{crit} — количество активных critical-алертов в подсистеме, N_{warn} — warning-алертов. Минимальный множитель — 0.5 (максимальный штраф — 50 %).

10.4 Оценка подсистемы

Каждый тип локомотива определяет набор подсистем (например, для KZ8A: Тяга, Электрика, Охлаждение, Тормоза, Вспомогательные). Оценка подсистемы sub :

$$S_{\text{sub}} = \text{penalty}(\text{sub}) \cdot \frac{\sum_{p \in \text{sub}} w_p \cdot \text{norm}(v_p, p)}{\sum_{p \in \text{sub}} w_p}$$

где w_p — вес параметра p внутри подсистемы (из конфигурации). Конкретные подсистемы и веса — см. `Locomotive_Metrics_Reference.md`.

10.5 Глобальный Health Index

$$HI = \left\lfloor \frac{\sum_{sub} W_{sub} \cdot S_{sub}}{\sum_{sub} W_{sub}} \cdot 100 \right\rfloor$$

где W_{sub} — вес подсистемы в глобальном индексе. Результат округляется до 1 знака после запятой и ограничивается $[0, 100]$.

10.6 Категоризация

Диапазон	Статус	Цвет	Описание
70 – 100	Normal		Все системы в безопасных диапазонах
40 – 69	Attention		Один или более параметров в warning-зоне
0 – 39	Critical		Требуется немедленное вмешательство

10.7 Объяснимость — Тор-5 факторов

После вычисления HI определяются 5 параметров с наибольшим негативным вкладом:

$$\text{contribution}(p) = W_{\text{sub}(p)} \cdot w_p \cdot (1 - \text{norm}(v_p, p))$$

Сортировка по убыванию, берутся top-5. Каждая запись содержит: название параметра, подсистему, текущее значение, безопасный диапазон, величину вклада, рекомендуемое действие.

10.8 Частота и производительность

HI пересчитывается на каждый тик телеметрии (1 Гц). Целевое время вычисления: < 50 мс на один локомотив. Результаты кешируются в оперативной памяти и записываются в `health_index_h`.

11 Спецификация REST API

11.1 Общие соглашения

- Base URL: `/api/v1/`
- Content-Type: `application/json`
- Аутентификация: Bearer JWT в заголовке `Authorization`
- Ошибки: `{"error": "code", "message": "...", "detail": {...}}`
- Пагинация: `?limit=N&offset=M`, заголовков ответа `X-Total-Count`
- Timestamps: ISO 8601 с timezone (UTC)

11.2 Authentication

Метод	Путь	Описание	Auth
POST	<code>/auth/login</code>	Аутентификация. Body: <code>{username, password}</code> . Ответ: <code>{token, role, expires_at}</code>	—
GET	<code>/auth/me</code>	Текущий пользователь: <code>{id, username, role}</code>	Required

11.3 Telemetry

Метод	Путь	Описание	Auth
GET	/telemetry/{loco_id}/latest	Последние значения всех параметров	dispatcher+
GET	/telemetry/{loco_id}/history	Исторические данные. Query: parameter, from, to, interval	dispatcher+
POST	/telemetry/ingest	Ручная отправка телеметрии (тестирование)	admin

11.4 Health Index

Метод	Путь	Описание	Auth
GET	/health-index/{loco_id}/current	Текущий HI + top factors + subsystem scores	dispatcher+
GET	/health-index/{loco_id}/history	История HI. Query: from, to	dispatcher+

11.5 Alerts

Метод	Путь	Описание	Auth
GET	/alerts/{loco_id}	Список алертов. Query: severity, from, to, limit, offset	dispatcher+
PATCH	/alerts/{id}/acknowledge	Подтвердить прочтение алерта	dispatcher+
PATCH	/alerts/{id}/annotate	Добавить отметку события. Body: {text}	dispatcher+

11.6 Configuration

Метод	Путь	Описание	Auth
GET	/config/{loco_type}	Конфигурация локомотива (параметры, пороги, веса)	dispatcher+
PUT	/config/{loco_type}	Обновить конфигурацию	admin
GET	/config/subsystems/{loco_type}	Список подсистем и их веса	dispatcher+

11.7 Simulator Control

Метод	Путь	Описание	Auth
POST	/simulator/start	Запустить симуляцию	admin
POST	/simulator/stop	Остановить симуляцию	admin
POST	/simulator/scenario	Сменить сценарий. Body: {scenario}	admin
GET	/simulator/status	Текущий статус симулятора	admin

11.8 Export

Метод	Путь	Описание	Auth
GET	/export/{loco_id}	Экспорт отчёта. Query: format=pdf csv, from, to	dispatcher+

11.9 System

Метод	Путь	Описание	Auth
GET	/health	Health-check (DB, queue, uptime)	—
GET	/docs	Swagger UI (авто-генерация)	—

11.10 WebSocket Protocol

Endpoint: `ws://host/ws/telemetry/{loco_id}`

1. Клиент подключается с JWT-токеном как query parameter (`?token=...`).
2. Сервер аутентифицирует и начинает стриминг.
3. Сервер отправляет JSON-фреймы с частотой 1 Гц:

```
{
  "type": "telemetry_update",
  "data": {
    "ts": "2026-04-04T12:00:00.000Z",
    "loco_id": "KZ8A-001",
    "parameters": { "speed_kmh": 87.3, ... },
    "health_index": { "score": 82.5, "status": "Normal", ... },
    "new_alerts": [ ... ]
  }
}
```

4. Клиент может отправить: `{"type": "ping"}`, сервер ответит `{"type": "pong"}`.
5. При отключении клиент реализует **exponential backoff**: $1\text{ с} \rightarrow 2\text{ с} \rightarrow 4\text{ с} \rightarrow 8\text{ с} \rightarrow \max 30\text{ с}$.
6. Визуальная индикация «нет связи» на UI в течение 3 с после потери соединения.
7. При реконнекте сервер сначала отправляет snapshot текущего состояния, затем возобновляет стриминг.

12 Описание экранных форм

12.1 Login Screen

Центрированная карточка на нейтральном фоне. Логотип проекта сверху, поля «Username» и «Password», кнопка «Войти». Toast-уведомление при ошибке. Автоматический редирект на Dashboard при наличии валидного токена. Минимальный дизайн, без sidebar.

12.2 Main Dashboard

- **Header** (full-width): логотип, селектор локомотива (dropdown), текущий пользователь + badge роли, переключатель темы, кнопка Logout.
- **Основная область**: Health Index hero-виджет (круговой gauge, число, текст статуса + цвет). Сетка панелей подсистем (Скорость, Топливо/Энергия, Давления/Температуры, Электрика). Каждая панель: мини-gauge + текущее значение + стрелка тренда (↑↓→).
- **Боковая/дополнительная область**: список Топ-5 факторов (параметр, bar вклада, значение vs порог). Лента алертов (scroll, последние 10). Рекомендации (текстовый список).
- **Нижняя полоса**: мини-график HI за последние 5 мин, индикатор соединения.

12.3 Alerts Panel

Полноширинная таблица: столбцы — Время, Параметр, Severity (иконка + текст), Значение, Порог, Статус, Аннотация. Над таблицей — фильтры (severity checkboxes, date range picker, селектор локомотива). Клик по строке — expand с деталями + текстовое поле аннотации. Пагинация внизу.

12.4 Route Map

Карта занимает основную часть экрана. Floating overlay: текущая скорость, координаты, ближайшая станция. Линия маршрута с цветовой кодировкой (зелёный/жёлтый/красный по HI). Маркер-иконка локомотива. Зоны ограничения скорости — dashed overlay.

12.5 Trends / Charts

Панель выбора диапазона: кнопки-пресеты (5 мин, 10 мин, 15 мин) + произвольный picker. Сетка графиков (настраиваемые метрики, количество и расположение — на усмотрение дизайнера). Каждый график: line chart, tooltip, авто-скейлинг Y, zoom через drag-select. Кнопка «Добавить метрику». Переключатель live/pause.

12.6 Replay / History

Timeline-скраббер (full-width) с кнопками Play / Pause / Speed (1×, 2×, 5×). Под скраббером — тот же layout, что и Dashboard, но данные управляются позицией курсора. Графики, карта, алерты, HI — всё синхронизировано со скраббером.

12.7 Admin Settings

Tabs-интерфейс:

- **Пороги**: таблица per locomotive type. Строки = параметры, столбцы = safe_low, safe_high, warn_low, warn_high, crit_low, crit_high, weight. Inline-editing с валидацией.

- **Симуляция:** кнопки Start/Stop, dropdown сценариев (normal, overheat, fault, highload, connection_loss), индикатор текущего статуса.
- **Пользователи:** простая CRUD-таблица (username, role, reset password).

13 Симулятор телеметрии

13.1 Normal Operation Mode

Random walk вокруг номинальных значений для каждого параметра, ограниченный safe-диапазонами. Гауссов шум с настраиваемой σ . Частота эмиссии — 1 Гц на локомотив.

13.2 Аварийные сценарии (минимум 3)

#	Сценарий	Длительность	Описание
1	Постепенный перегрев	2 мин	Температура тягового двигателя дрейфует вверх, проходя warning → critical зоны.
2	Внезапный электрический сбой	мгновенно	Напряжение падает до 0 за 1 тик. Множественные алерты.
3	Highload burst	30 сек	Частота сообщений увеличивается до $\times 10$. Тест backpressure.

13.3 Connection Loss

Симулятор прекращает отправку на 10–15 секунд, затем возобновляет. Frontend должен отобразить «нет связи» и корректно восстановиться.

13.4 Конфигурация

Сценарии описываются в YAML/JSON файлах. Каждый сценарий определяет: затронутые параметры, тип изменения (drift/step), скорость, длительность, целевые значения.

14 Нефункциональные требования

14.1 Производительность / Latency

- End-to-end latency (от эмиссии события до видимого обновления UI): < 500 мс.
- Вычисление Health Index: < 50 мс на локомотив на тик.
- Время ответа API (исторические запросы, 1-часовой диапазон): < 200 мс.
- Frontend-рендеринг: 60 fps в нормальном режиме.

14.2 Highload / Burst

- Система выдерживает $\times 10$ burst (10 Гц вместо 1 Гц) без аварийной остановки.
- Очередь буферизирует всплески; backend обрабатывает на устойчивой скорости.
- Frontend может снизить частоту рендеринга до 2–4 fps при burst, но **не должен зависать**.
- Graceful degradation: при отставании пропускаются промежуточные фреймы, всегда отображается последнее состояние.

14.3 Отказоустойчивость

- WebSocket reconnect с exponential backoff (1, 2, 4, 8, max 30 с).
- Визуальная индикация «нет связи» на frontend в течение 3 с после disconnect.
- Backend: при временной недоступности БД — отдача данных из in-memory кеша до 30 с.
- Нет потери данных в очереди при перезапуске backend (Redis persistence или graceful shutdown asyncio.Queue).

14.4 Безопасность

- JWT-аутентификация для всех API endpoints (кроме /health, /docs).
- Role-based access control: Admin — полный доступ, Dispatcher — read-only, Cabin — read-only subset.
- Секреты (пароль БД, JWT secret) — **только в переменных окружения**, никогда в коде.
- Все данные — mock/обезличенные.

14.5 Документация API

- Авто-генерируемая документация OpenAPI 3.0 / Swagger UI на /docs.
- Все endpoints задокументированы: request/response schemas, примеры, коды ошибок.

14.6 Инфраструктура

- **Docker Compose** — единственный способ запуска. `docker-compose up` поднимает все сервисы: simulator, backend, frontend, TimescaleDB, Redis.
- Конфигурация через `.env` файл с документированными переменными.
- `.env.example` в репозитории с описанием всех переменных.
- `README.md` с пошаговыми инструкциями запуска.

14.7 Логи и наблюдаемость

- Структурированные JSON-логи от всех backend-сервисов (`structlog`).
- Уровни логирования: DEBUG, INFO, WARNING, ERROR.
- Request logging с `trace_id` для корреляции.
- Endpoint /health — статус сервиса, связность БД, глубина очереди, uptime.
- Метрики: активные WebSocket-соединения, msg/sec, queue depth, HI computation time.

15 План тестирования

15.1 Unit Tests

ID	Компонент	Тест	Ожидаемый результат
UT-01	HI Engine	Нормализация: значение в safe-зоне	Возвращает 1.0
UT-02	HI Engine	Нормализация: значение в warning-зоне	Пропорциональная оценка (0, 1)
UT-03	HI Engine	Нормализация: значение за critical	Возвращает 0.0

ID	Компонент	Тест	Ожидаемый результат
UT-04	HI Engine	Нормализация: значение NaN	Возвращает 0.0
UT-05	HI Engine	Штраф за алерты	Корректный мультипликативный штраф
UT-06	HI Engine	Топ-5 факторов: порядок	По убыванию contribution
UT-07	HI Engine	Полный расчёт HI (профиль KZ8A)	Результат $\in [0, 100]$
UT-08	Ingest	Дедупликация duplicate timestamp	Второе сообщение отброшено
UT-09	Ingest	Отклонение выброса	Значение вне физического диапазона отвергнуто
UT-10	Ingest	ЕМА-сглаживание	Сглаженное значение соответствует $\alpha = 0.3$

15.2 Integration Tests

ID	Тест	Шаги	Ожидаемый результат
IT-01	Pipeline end-to-end	POST телеметрию → GET history	Данные в БД в течение 1 с
IT-02	WebSocket соединение	Подключиться, получить данные	JSON-фреймы с частотой 1 Гц
IT-03	WebSocket reconnect	Разорвать соединение, ждать	Клиент переподключается с backoff
IT-04	Auth flow	Login с корректными creds	JWT-токен, доступ к protected endpoint
IT-05	Auth rejection	Запрос без токена	Ответ 401 Unauthorized
IT-06	Role enforcement	Dispatcher → PUT config	Ответ 403 Forbidden
IT-07	Alert generation	Отправить значение выше порога	Алерт создан, отправлен через WS
IT-08	Config update → HI	Изменить порог, проверить HI	HI пересчитан с новыми порогами

15.3 Scenario / E2E Tests

ID	Сценарий	Время	Критерий успеха
ST-01	Normal operation	5 мин	Dashboard работает, HI стабильно 80–95, нет ложных алертов
ST-02	Постепенный перегрев	3 мин	HI плавно снижается, алерты появляются, top factors показывают температуру
ST-03	Внезапный сбой	1 мин	HI резко падает, статус Critical, множественные алерты, рекомендации
ST-04	Highload burst $\times 10$	1 мин	UI не зависает, данные eventually consistent
ST-05	Connection loss + recovery	2 мин	«Нет связи» через 3 с, авто-reconnect, данные возобновляются
ST-06	Replay mode	2 мин	Историческое воспроизведение с синхронизацией графиков/карты/алертов
ST-07	Report export	1 мин	PDF/CSV скачивается с корректными данными
ST-08	Переключение темы	30 с	Все экраны корректно отображаются в обеих темах
ST-09	Смена типа локомотива	1 мин	Переключение KZ8A $\leftrightarrow$ ТЭ33А, параметры обновляются

15.4 Performance Tests

ID	Тест	Целевое значение	Инструмент
PT-01	WebSocket latency (end-to-end)	< 500 мс	Сравнение timestamp
PT-02	API response (history, 1 ч)	< 200 мс	cURL timing
PT-03	Burst handling ($\times 10$, 30 с)	Без crash	Simulator burst mode

16 Формат сдачи и демонстрация

16.1 Git-репозиторий

- Публичный или общий Git-репозиторий.
- Чистая история (без секретов в коммитах).
- README.md: описание проекта, обзор архитектуры, prerequisites (Docker, docker-compose), пошаговая инструкция запуска (docker-compose up), учётные данные для демо-входа, скриншоты/GIF работающего приложения.
- .env.example: все необходимые переменные окружения с документацией.

16.2 Презентация

10–12 слайдов:

1. Постановка проблемы
2. Архитектурная диаграмма
3. Разбор формулы Health Index
4. Обоснование выбора стека
5. Демо-скриншоты ключевых экранов
6. Вызовы и решения

Формат: PDF или Google Slides.

16.3 Демонстрация

Live-демо или предзаписанное видео (3–5 минут), демонстрирующее:

- Нормальный режим работы с live-dashboard.
- Аварийный сценарий с деградацией НИ и алертами.
- Highload burst ($\times 10$).
- Потеря и восстановление соединения.
- Переключение темы.
- Replay mode.
- Смена типа локомотива.

16.4 Архитектурная диаграмма

Отдельный файл (PNG/SVG) в `docs/architecture.png`. Содержит все компоненты, потоки данных, порты и протоколы (см. раздел 8.2).

17 Definition of Done

1. Все сервисы поднимаются через единый `docker-compose up`.
2. KZ8A и ТЭ33А поддерживаются через отдельные конфигурационные профили (без изменения кода).
3. Swagger/OpenAPI документация доступна на `/docs`.
4. Frontend отображает: dashboard с НИ, панели узлов, карту маршрута, trends, replay.
5. Светлая и тёмная тема полностью функциональны.
6. Health Index считается в реальном времени с объяснимостью (top-5 факторов).
7. Минимум 3 демо-сценария подготовлены (normal, fault, highload/connection loss).
8. WebSocket reconnect с backoff + визуальная индикация «нет связи».
9. JWT-аутентификация с role-based access control.
10. Endpoint `/health` возвращает статус сервиса.
11. Структурированное JSON-логирование включено.
12. Экспорт отчёта (PDF/CSV) функционален.
13. Отметки событий (аннотации к алертам) реализованы.
14. Презентация (10–12 слайдов) с архитектурой и формулой НИ.
15. `README.md` с полными инструкциями запуска.

18 Глоссарий

Термин	Определение
HI	Health Index — интегральный показатель состояния локомотива (0–100)
EMA	Exponential Moving Average — экспоненциальное скользящее среднее
KZ8A	Электровоз казахстанского производства (переменный ток 25 кВ)
ТЭ33А	Тепловоз с дизель-электрической тягой (GE Evolution)
NFR	Non-Functional Requirements — нефункциональные требования
MVP	Minimum Viable Product — минимально жизнеспособный продукт
SSE	Server-Sent Events — односторонний серверный push через HTTP
WebSocket	Двухнаправленный протокол связи поверх TCP
TimescaleDB	Расширение PostgreSQL для работы с time-series данными
Hypertable	Таблица TimescaleDB с автоматическим партиционированием по времени
JWT	JSON Web Token — стандарт аутентификации на основе подписанных токенов
WCAG	Web Content Accessibility Guidelines — стандарт доступности веб-контента
RBAC	Role-Based Access Control — управление доступом на основе ролей

19 Приложения

Приложение А. Параметры локомотивов

Полная таблица телеметрических параметров для KZ8A и ТЭ33А (названия, единицы измерения, подсистемы, safe/warn/crit диапазоны, веса) — см. *Locomotive_Metrics_Reference.md*.

Приложение В. Формат JSON-сообщения телеметрии

```
{
  "loco_id": "KZ8A-001",
  "loco_type": "KZ8A",
  "ts": "2026-04-04T12:00:00.000Z",
  "parameters": {
    "speed_kmh": 87.3,
    "traction_motor_temp_c": 142.5,
    "voltage_v": 24800,
    "current_a": 350,
    "brake_pressure_bar": 4.8,
    "oil_temp_c": 78.2,
    "coolant_temp_c": 82.0,
    "fuel_level_pct": 65.0
  }
}
```

Приложение С. Формат WebSocket фрейма

```
{
  "type": "telemetry_update",
  "data": {
```

```

    "ts": "2026-04-04T12:00:01.000Z",
    "loco_id": "KZ8A-001",
    "parameters": { ... },
    "health_index": {
      "score": 82.5,
      "status": "Normal",
      "top_factors": [
        {
          "parameter": "traction_motor_temp_c",
          "subsystem": "Traction",
          "value": 142.5,
          "safe_range": [80, 130],
          "contribution": 0.15,
          "action": "Monitor temperature trend"
        }
      ],
      "subsystem_scores": {
        "Traction": 0.78,
        "Electrical": 0.95,
        "Cooling": 0.88,
        "Braking": 0.92
      }
    },
    "new_alerts": []
  }
}

```

Приложение D. Docker Compose — обзор сервисов

Сервис	Образ	Порт	Зависимости
frontend	Node 20 + Next.js	3000	backend
backend	Python 3.11 + FastAPI	8000	db, redis
simulator	Python 3.11	—	redis
db	timescale/timescaledb	5432	—
redis	redis:7-alpine	6379	—

Приложение E. Цветовая палитра

Конкретные цвета выбираются командой дизайна. Общие требования к палитре:

- Поддержка светлой и тёмной темы с достаточным контрастом (WCAG AA).
- Семантические цвета статусов: зелёный (Normal), жёлтый/оранжевый (Attention), красный (Critical) — должны быть интуитивно различимы.
- Единый акцентный цвет для интерактивных элементов.
- Иерархия текста: primary и secondary цвета для основного и вспомогательного текста.
- Нейтральные фоновые и поверхностные цвета для карточек, панелей и разделителей.